Capítulo

8

Optimización

En todos los programas de la materia Base de datos se da una constante clara: la optimización de las consultas. Si bien es una disciplina lo suficientemente importante como para formar especialistas, a veces se comete el error de no verlo desde un punto de vista ambiental. Es decir pensar un tanto "out of the box". Quizás exista un profesional dedicado a la optimización de las consultas que realiza un sistema transaccional, cuando en realidad el cuello de botella del sistema se hallaba en el ancho de banda de las conexiones. De este modo se desperdician valiosos recursos.

No es lo mismo optimizar una base de datos para un OLAP, que optimizar un proceso batch, o una consulta que se realiza en forma concurrente en un sistema OLTP. Es por eso que, si bien nos centraremos en la optimización de consultas en una base de datos, veremos también cómo optimizar sus recursos según el escenario que se presenta.

Optimización de la performance

La performance no debe ser optimizada una vez que el sistema ha sido puesto en marcha, debe tenerse en cuenta desde la etapa de diseño, sea cuál sea la naturaleza del mismo (OLTP, OLAP, Batch, etc.).

Tiempo de respuesta y Throughput

Cómo remarcamos anteriormente, las metas de la optimización varían según la naturaleza y necesidades de la aplicación que va a optimizarse. Las aplicaciones OLTP definen su performance en términos de throughput, es decir en Transacciones (u operaciones) por unidad de tiempo. Estas aplicaciones deben procesar miles o quizás millones de transacciones relativamente pequeñas por día.

En contraste, las aplicaciones de soporte de toma de decisiones (Datawarehouse, OLAP, DSS) definen la performance en términos de tiempo de respuesta. Las demandas que se le puede hacer a este tipo de sistemas varían drásticamente, en un momento quizás una consultas devuelva unos pocos registros, pero en otro momento puede requerirse procesamiento paralelo masivo que retraiga y ordene cientos de miles de registros de varias tablas diferentes.

Cómo optimizar tiempo de respuesta

Cómo el tiempo de respuesta se puede definir matemáticamente cómo:

$$tr = ts + te$$

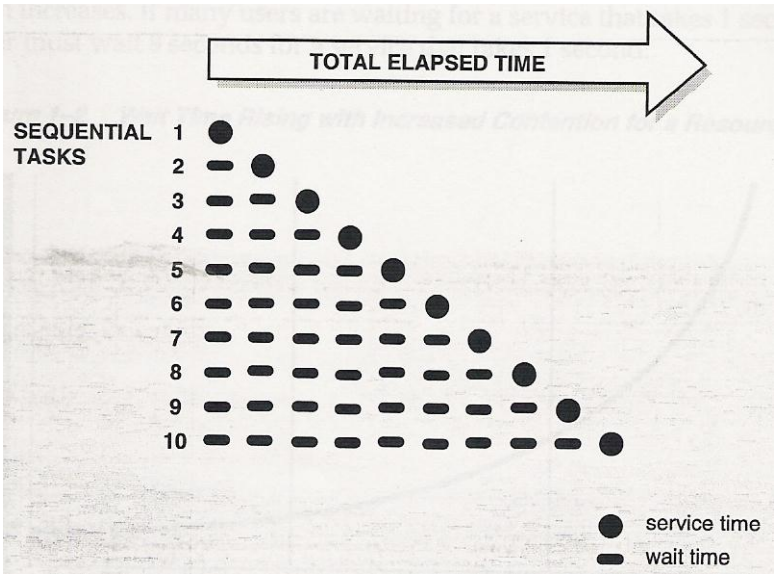
dónde:

- *tr es tiempo de respuesta*
- *ts es tiempo de servicio*
- *te es tiempo de espera*

Podemos decir que se puede optimizar el tiempo de respuesta mediante una de las dos opciones siguientes:

- Reduciendo el tiempo de Servicio
- Reduciendo el tiempo de Espera

Podemos graficar qué ocurre con una serie de tareas anidadas en un procesador. Obviamente si contamos con varios procesadores, contaremos con paralelismo y este gráfico aplicará a uno sólo de los procesadores en cuestión. Se ve claramente cómo la tarea 2 debe esperar a que termine la tarea 1, la 3 a la 2 y así sucesivamente hasta que la tarea 10 espera a que terminen todas.



Cómo optimizar el Throughput del Sistema

Cómo dijimos previamente, el throughput del sistema consiste en la cantidad de trabajo realizado en una cantidad de tiempo dada. Matemáticamente podemos expresar:

$ts = w/t$

dónde:

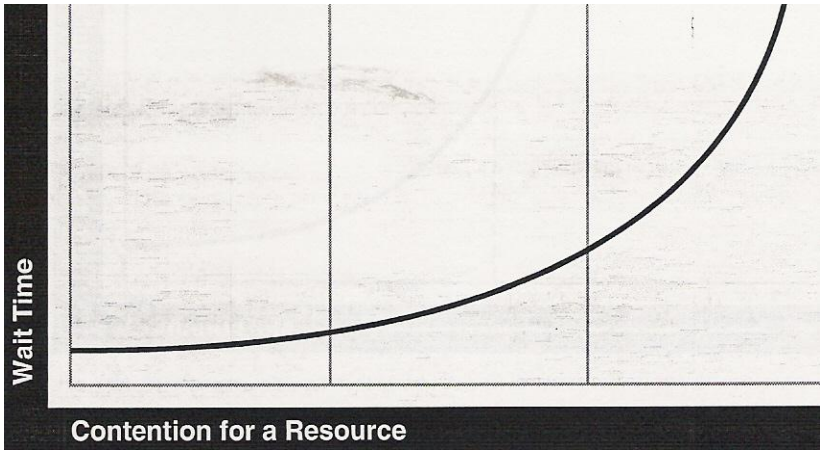
- ts es el throughput del sistema
- w es la cantidad de trabajo realizada
- t el tiempo transcurrido

Por lo tanto para optimizarlo, debemos utilizar alguna (o una combinación) de las siguientes técnicas:

- Hacer más trabajo con los mismo recursos (reducir el tiempo de servicio)
- Hacer el trabajo más rápido, reduciendo el tiempo de respuesta general. Para hacer esto, hay que prestar atención al tiempo de espera. Por ejemplo, agregando 1 CPU.

Tiempo de Espera

El tiempo de espera para una tarea puede permanecer constante, pero crecerá si la contención de recursos crece. Por ejemplo si muchos usuarios están esperando por un servicio que toma 1 segundo ejecutar, el décimo usuario, esperará 10 segundos. Es por eso que hay que evitar los cuellos de botella en los recursos críticos.



Recursos críticos:

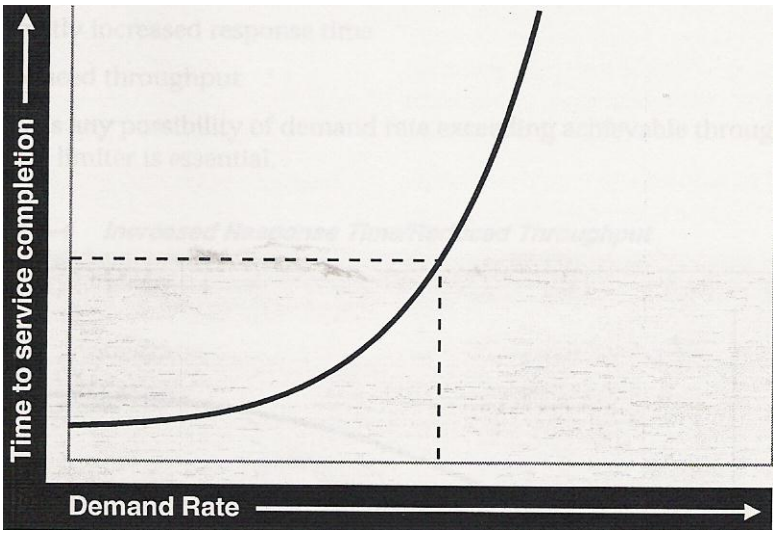
- CPUs
- Capacidad de I/O
- Ancho de banda

Por eso la performance dependerá de:

- Cuántos recursos están disponibles

- Cuántos clientes necesitan el recurso
- Cuánto deben esperar por un recurso
- Por cuánto tiempo "capturan" un recurso

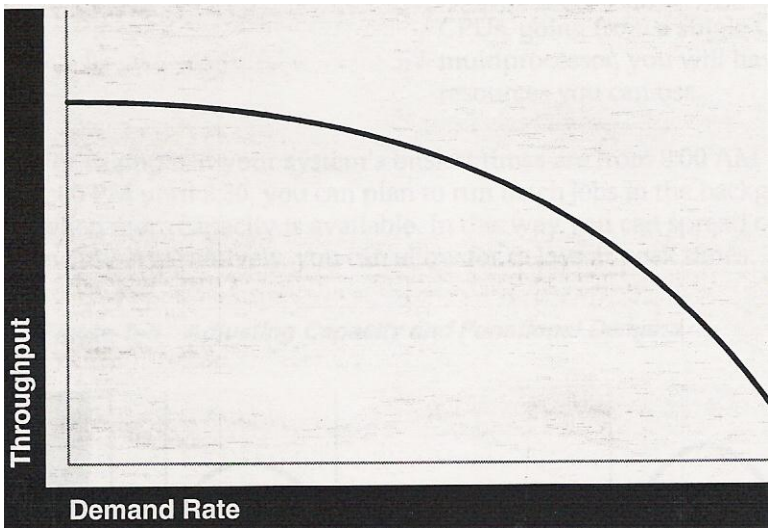
Por tanto el tiempo de servicio, según la capacidad de los recursos utilizados, dependerá de la taza de demanda de uso de los recursos cómo se ve en el siguiente gráfico:



Para manejar esta situación, se puede:

- Limitar la demanda para mantener tiempos de respuesta aceptables
- Agregar recursos (cpu, discos, memorias)

Siempre es conveniente prevenir estas situaciones, ya que los efectos de la demanda excesiva suelen ser nocivos para la salud de los sistemas, cómo puede apreciarse en el siguiente gráfico:



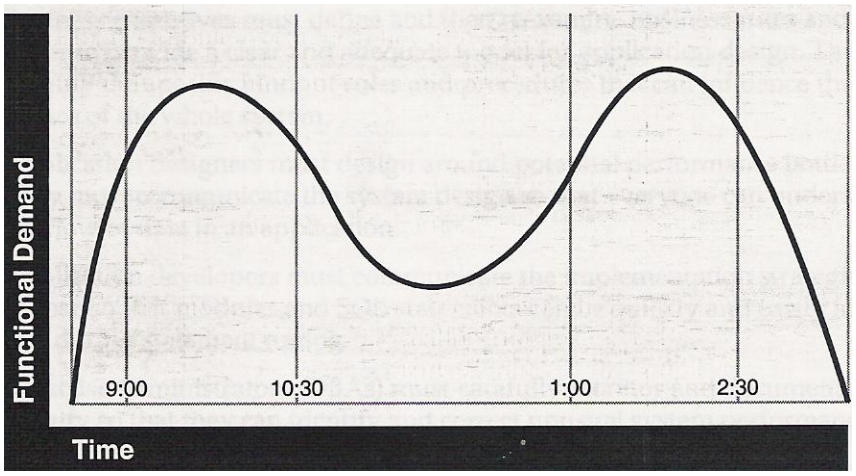
- Incremento importante del tiempo de respuesta

- Throughput reducido

Existen ciertos ajustes básicos que pueden hacerse para aliviar los problemas de performance:

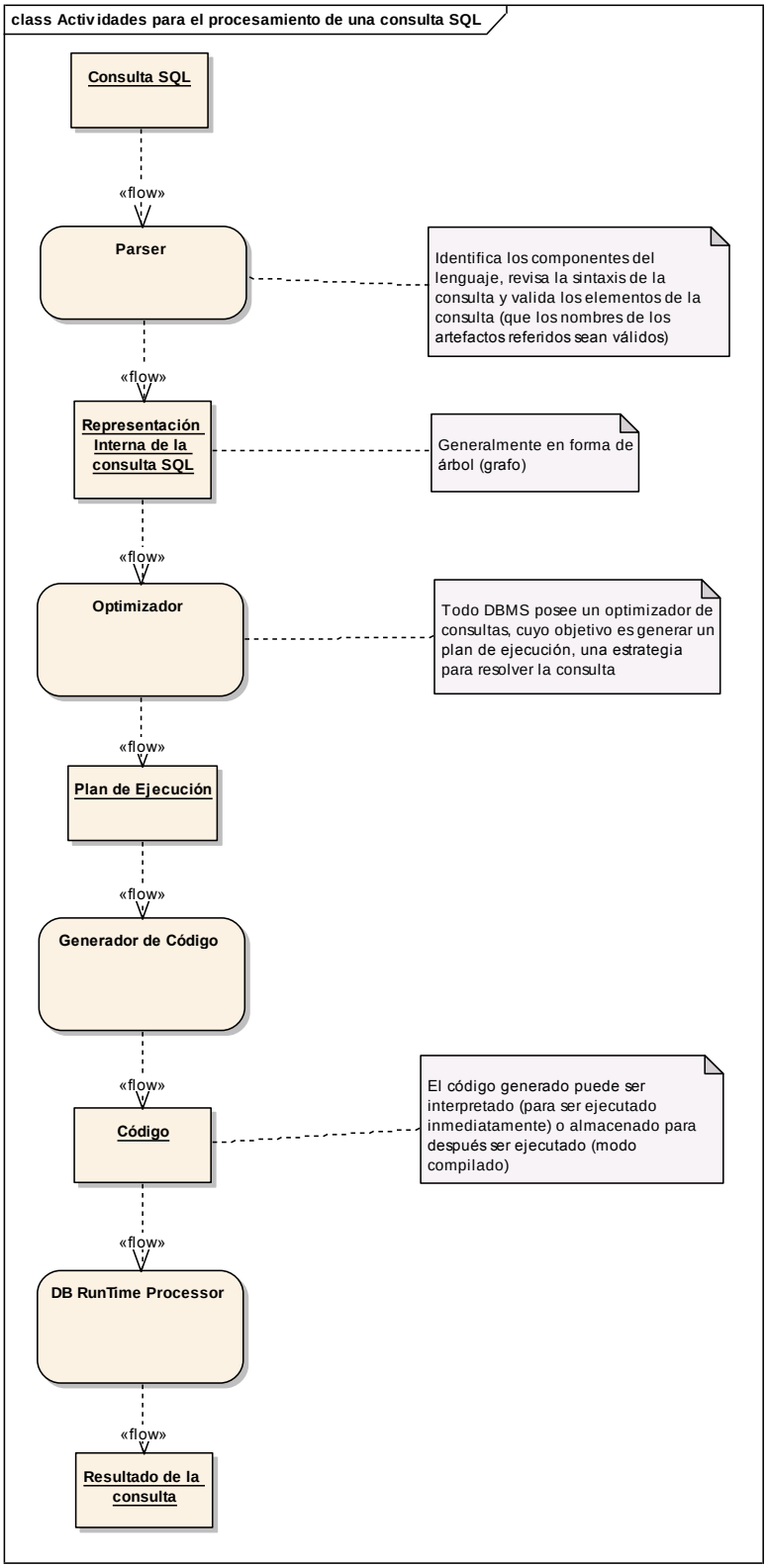
- *Ajustar el consumo unitario:* Usando menos recursos por transacción o reduciendo el tiempo de servicio. También pueden reducirse el número de operaciones de I/O por transacción.
- *Ajustar la demanda funcional*: Re planificando o redistribuyendo el trabajo.
- *Ajustando la capacidad*: Incrementando o re-alocando recursos.

Viéndolo desde un ejemplo práctico, podemos pensar que los momentos del día en el que nuestro sistema está más ocupado son desde las 9:00am hasta las 10:30, y desde las 13:00 hasta las 14:30 es cuándo hay más capacidad disponible. Quizás se pueda redistribuir el trabajo para que ciertos trabajos "batch" corran en la franja más ociosa y por otro lado podamos de alguna manera distribuir más la demanda en los picos, permitiéndonos ciertos delays.



Enfocándonos en el DBMS

Las actividades que realiza un DBMS para procesar una consulta SQL se pueden diagramar de la siguiente forma:



Existen dos técnicas fundamentales para optimizar consultas que puede utilizar un optimizador:

- *Reglas*: Se basa en reglas heurísticas para ordenar las operaciones para resolver la consulta.
- *Costo*: Se basa en la estimación sistemática del costo de diferentes estrategias de ejecución.

Ambas estrategias suelen combinarse en el optimizador.

Algoritmos básicos para la ejecución de consultas

- Operación de **selección**: A estos algoritmos se los conoces cómo examinadores de archivos (si implica la utilización de un índice se le llama examinador de índices):
    1. *Búsqueda lineal*: por fuerza bruta se evalúan todas la tuplas.
    2. *Búsqueda binaria*: si el archivo está ordenado por el criterio por el cual se quiere seleccionar y el criterio es una igualdad, se utiliza este tipo de búsqueda.
    3. *Uso de una PK*: si el criterio implica una comparación de igualdad sobre un atributo que es clave primaria (o que pertenece a la clave primaria y es el primero en ella), se utiliza el índice, reduciendo la complejidad a la indicada por el índice.
    4. *Uso de una PK para múltiples registros*: si el criterio implica una comparación de <, <=, > o >=, igualdad sobre un atributo que es clave primaria (o que pertenece a la clave primaria y es el primero en ella), se utiliza el índice por rangos, reduciendo la complejidad.
    5. *Uso de un índice secundario*: Es el mismo caso que los dos anteriores, pero si los criterios se aplican sobre un atributo que tenga específicamente creado un índice.

Si la condición es conjuntiva (dos o más condiciones unidas por AND), también se pueden utilizar los siguientes métodos:

- *Selección conjuntiva*: Si una de las conjunciones puede resolverse vía los métodos del 2 al 5, primero se resuelve es conjunción, reduciendo el conjunto de tuplas procesadas, y luego sobre el conjunto resultante se emplea la misma técnica hasta que queden resueltas las conjunciones o haya que utilizar el método 1.
- *Selección conjuntiva con un índice compuesto*: Si se determina que 2 o más de las conjunciones matchea con la definición de un índice compuesto (más de un atributo), se utilizará ese índice para reducir el conjunto de operaciones. Luego se continuará resolviendo las conjunciones sucesivamente.
- *Selección conjuntiva por intersección de punteros a registros:* Si se dispone de índices secundarios (que apliquen a la consulta) que utilicen punteros a registros (no a bloques), en los casos que el criterio sea de igualdad, se pueden usar dichos índices para obtener los punteros a registros para cada criterio, de modo que se pueda aplicar la intersección entre esos conjunto de punteros para determinar cuáles son los registros válidos.

Selectividad de una condición atómica: Se define cómo la razón entre la cantidad de tuplas (o registros de ahora en más) que satisfacen la condición, y el número total de registros de la tabla (relación).

Mientras menor sea la selectividad de una condición, menor cantidad de registros que seleccionará y más deseable será procesar primero esa condición.

La información sobre selectividad se obtiene del catálogo de la base de datos, dónde quizás puede no estar la información exacta por lo que se almacenan estimaciones de selectividad que son aprovechadas por el optimizador.

- Operación de **junta (join)**: Esta operación es una de las más costosas que debe resolver el DBMS.
    - *Nested Loops (ciclos anidados):* Por fuerza bruta, por cada registro de uno de los conjuntos, se evaluarán todos los registros del otro. De ese modo la cantidad de registros a evaluar es el producto de las cantidades de registros de cada conjunto que participa en el join.
    - *Index Nested Loops*: Similar al anterior, sólo que en el caso en que se pueda utilizar un índice en alguna de las tablas participantes en el join, se accederá a esa parte de la junta vía el índice, reduciendo la complejidad.
    - *Sort-Merge Join*: Si las dos tablas del join tienen un índice clusterizado por el atributo de junta, se podrá implementar el sort-merge join de la forma más eficiente:
        - Se examinan ambos archivos en orden, buscando los registros que tienen los mismos valores en ambas tablas, de este modo se logra un apareo por el cuál un registro es examinado una sola vez para compararlo con el otro.

      Variación para índices secundarios: Si en vez de ser índices tipo cluster, nos encontramos con índices secundarios, también se puede hacer este apareo, pero teniendo en cuenta que quizás los registros estén dispersos por muchos bloques, por lo cual puede no ser muy eficiente.
    - *Hash-Join*: Los registros de ambas tablas se dispersan al mismo archivo de dispersión, usando la misma función de hash, con los atributos de join cómo claves del hash. Una sola pasada por la tabla con el menor número de registros dispersa sus registros a buckets del archivo de dispersión. Una sola pasada por el otro archivo dispersa entonces sus registros a la cubeta apropiada, dónde el registro se combina con todos los registros coincidentes de la primera tabla.

- Operación de **proyección**: Esta operación es la más simple de implementar, ya que si no es necesario eliminar duplicados, se traerá la lista de atributos necesarios (accediendo a los archivos).

- Operaciones de manejo de conjuntos:
    - Producto cartesiano: Es una de las operaciones más caras de realizar, ya que se deberán traer T1xT2 registros (suponiendo el producto cartesiano entre las tablas T1 y T2), y At1+At2 atributos (suponiendo At1 la cantidad de atributos pertenecientes a la tabla T1 y At2 su correspondencia en la tabla T2).
    - Unión: Adicionalmente al costo de evaluar las dos consultas que une la sentencia UNION, se hará una pasada extra por ambos conjuntos, tanto para retraerlos, cómo para eliminar duplicados.
    - Unión Aditiva: Igual que la unión, sólo que no se eliminan duplicados.
    - Intersección: Se utiliza un esquema similar al del Hash-join, se dispersa el conjunto 1 y luego al dispersarse el conjunto 2 (teniendo en cuenta todos los atributos siempre), sólo se tomarán los coincidentes.
    - Diferencia de conjuntos: De forma similar a la intersección, se dispersarán ambos conjuntos, sólo que en vez de tomar los coincidentes en la segunda pasada, se los eliminará.

No es recomendable tener una rutina de acceso individual para cada operación, ya que las operaciones suelen darse en forma combinada (una selección, proyección, junta, etc.). Es por eso que los DBMS tienen estrategias para combinar estos (y otros algoritmos propietarios).

Heurística en la optimización

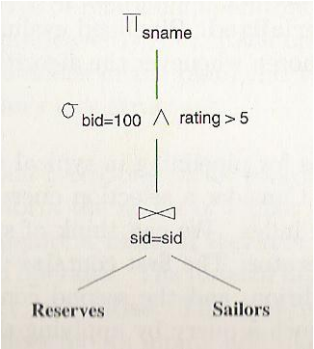
La principal regla heurística que utiliza el optimizador consiste en aplicar primero las operaciones de Selección y Proyección antes que las juntas u otras operaciones binarias, dado que el tamaño del conjunto resultante de una función binaria, es una función de los tamaños de los conjuntos de entrada. En cambio las operaciones de selección y proyección nunca aumentan el conjunto de entrada.

Un árbol de consulta es una estructura correspondiente a una expresión del álgebra relacional. En general, se pueden expresar muchos árboles de consulta para una misma expresión de álgebra relacional.

Ejemplo de árbol de consulta: Dado el siguiente query SQL:

```
select   S.sname
from     Reserves R inner join Sailors S on R.sid=S.sid
where    R.bid=100 and S.rating > 5;
```

El árbol de operaciones del álgebra relacional para ilustrar esta consulta, podría construirse de la siguiente manera:



Existen reglas generales de transformación para operaciones de álgebra relacional, con el fin de transformar una serie de operaciones en otra más óptima, por ejemplo:

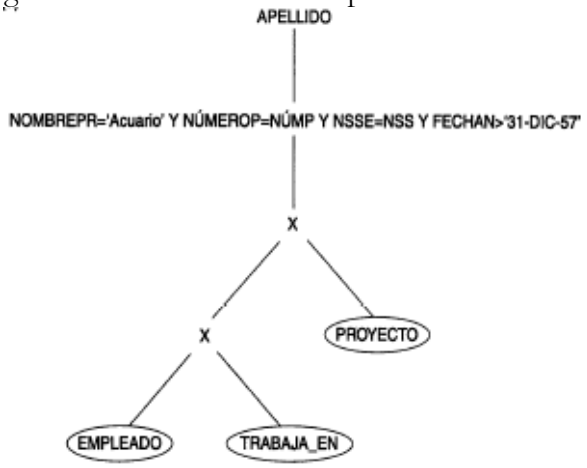
1. Cascada de selección: una condición de selección conjuntiva puede descomponerse en una secuencia de operaciones de selección individuales.
2. Conmutatividad de la operación selección consigo misma.
3. Cascada de proyección: en una secuencia de operaciones de proyección, podemos obviar todas menos la última.
4. Conmutación de selección con proyección: si en una condición de selección, sólo intervienen los atributos referidos en la proyección, se pueden conmutar ambas operaciones.
5. Conmutatividad de los join (junta).
6. Conmutación de la selección con el join: si todos los atributos de la condición de selección pertenecen a sólo una de las relaciones en la junta, las dos operaciones pueden conmutarse de la siguiente manera:
    - *selección c (R join S) = (selección c (R)) join S*
    - O bien, si la condición de selección c puede escribirse cómo (c1 and c2), y en la condición c1 intervienen sólo atributos de la tabla R, y en la condición 2, intervienen atributos de la tabla S únicamente, las operación pueden conmutarse cómo sigue:
      seleccion c (R join S) = (selección c1 (R)) join (selección c2 (S))
7. Conmutación de la selección con el producto cartesiano: ídem punto anterior.
8. Conmutación de la proyección con el join: de forma similar a la selección con las condiciones, pueden separarse los atributos en la proyección determinando qué lista de atributos se corresponde con la primer tabla y que lista con la segunda, teniendo siempre en cuenta los atributos de condición de junta, se puede hacer conmutable la proyección con la junta.
9. Conmutación de la proyección con el producto cartesiano: ídem punto anterior.
10. Conmutatividad de las operaciones de conjunto: las operaciones de unión e intersección con conmutables, pero la diferencia de conjuntos no lo es.
11. Asociatividad de junta, producto cartesiano, unión e intersección: todas estas operaciones con asociativas entre ellas.
12. Conmutatividad de la unión con la proyección

13. Conmutatividad de la selección con las operaciones de conjuntos: la selección es conmutable con la unión, intersección y diferencia de conjuntos.
14. Aplicación de las leyes de DeMorgan en las condiciones de selección

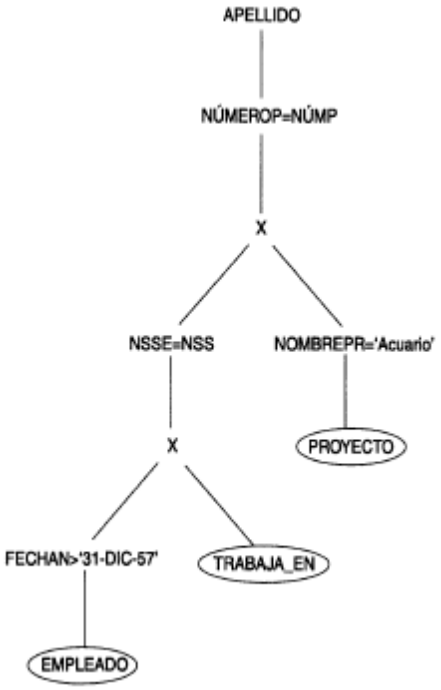
Los pasos del algoritmo para producir mejoras en árboles de consultas utilizando las reglas de transformación, podrían resumirse de la siguiente manera:

1. Utilizando la regla 1, descomponer cualquier operación de selección que tenga condiciones conjuntivas, de modo de poder acomodar en el árbol las operaciones de selección por las diversas ramas del árbol.
2. Aplicando las reglas 2, 4, 6, 7, 13, desplazar cada operación de selección tan abajo en el árbol cómo sea posible (cómo permitan los atributos de la condición).
3. Aplicando la regla 11, reacomodar los nodos hoja del árbol, de modo que las relaciones de nodo hoja con las operaciones de selección más restrictivas se ejecuten primero.
4. Intentar combinar una operación de producto cartesiano con una operación de selección de la que se pueda extraer de su condición una condición de reunión, de modo de convertir ambas operaciones en una junta.
5. Empleando las reglas 3, 4, 8, 9, 12, descomponer la lista de atributos de las operaciones de proyección y desplazarlas lo más abajo posible en el árbol creando nuevas operaciones proyección cuándo sea necesario.

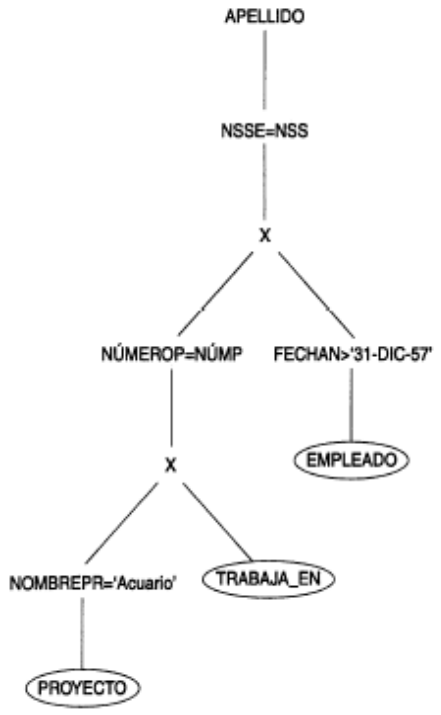
Ejemplo: Dado el siguiente árbol canónico de operaciones relacionales:



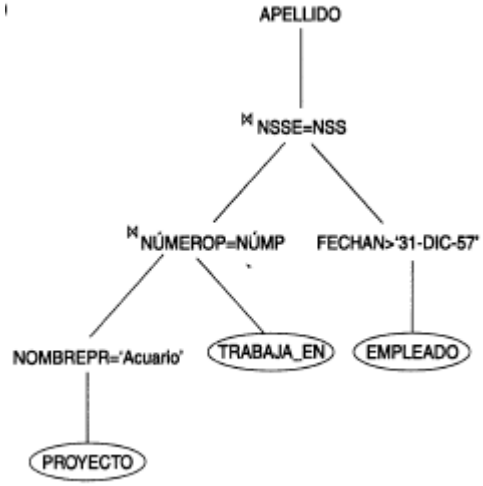
Aplicamos los pasos 1 y 2 del algoritmo, de modo que el árbol queda así:



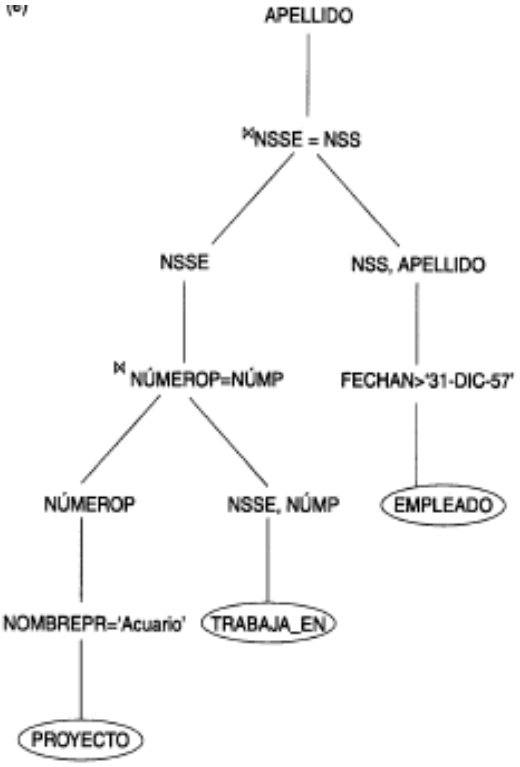
Luego de la aplicación del paso 3, el árbol quedará así:



Aplicando el paso 4:



Luego del paso 5:



Estimaciones de costo

Un optimizador no sólo se basa en reglas, sino también estima el costo de las estrategias de acceso a los datos para optimizar. El optimizador también emplea algoritmos para descartar estrategias de consultas a priori para no ser evaluadas. El enfoque de optimización por costo es más eficiente en consultas compiladas, ya que se evalúa el costo una vez, en cambio en consultas interpretadas, puede que se haga muy lenta la respuesta.

El costo de ejecutar una consulta consiste en:

1. Costo de acceso a almacenamiento secundario
2. Costo de almacenamiento de archivos intermedios
3. Costo de cómputo
4. Costo de comunicación

Información en el catálogo para la estimación de costo

El optimizador de consultas obtendrá la información necesaria para calcular el costo de una consulta del catálogo de la base de datos. Entre los datos que están almacenados, puede contarse con:

- Número de registros por tabla
- Número de bloques
- Estimaciones aproximadas de los dos anteriores cuándo no se pueda disponer del número exacto.
- Método de acceso primario y atributos de acceso primario
- Atributos de acceso secundario
- Si existe índice tipo cluster
- Número de niveles de cada índice multinivel
- Número de valores distinto de un atributo indexado (cardinalidad de la selección o número medio de registros que satisfarán una condición de selección sobre dicho atributo)

Cómo se verá hay atributos que no cambian muy a menudo de valores (número de niveles de un índice multinivel), pero otros si cambian constantemente.

Ejemplos de funciones de costo para la operación selección

1. Enfoque de búsqueda lineal: por fuerza bruta, examina todos los registros/bloques del archivo para obtener los que satisfagan la condición de selección, por lo tanto C(sel)=r. Si la condición es una igualdad, en promedio se evaluarán la mitad de los registros, o sea C(sel)=r/2.
2. Búsqueda binaria: Aproximadamente $C(sel)=Log_2 r$.
3. Empleo de un índice primario para obtener un sólo registro: C(sel)= x + 1 (dónde x es el número de niveles). Si se trata de un hash, C(sel) = 1.
4. Empleo de un índice para obtener múltiples valores (rangos): Aproximadamente en promedio la mitad de los registros satisfarán la condición, por lo que C(sel)= x + r/2.
5. Empleo de un índice clusterizado para obtener múltiples registros: Suponiendo que la condición de igualdad satisface unos s registros (cardinalidad de la condición de selección), C(sel)= c + x.
6. Empleo de un índice árbol B*: si la condición es de igualdad, y la selectividad de la condición es s, C(sel) = x + s. Si la condición es de rango, C(sel) = x + r/2.

Ejemplos de funciones de costo para la operación join

Definimos selectividad de la operación join (sj) entre las tablas R y S, de la siguiente manera:

$$sj = | (R \text{ join } S) | \ / \ | (R \times S) | = | (R \text{ join } S) | \ / \ (|R| . |S|)$$

$$0 <= sj <= 1$$

De este modo el optimizador puede estimar a priori el tamaño del resultado de un join vía la fórmula:

$$| R \text{ join } S | = sj . |R| . |S|$$

- Nested Loops: suponiendo que se utiliza R para el ciclo superior (o externo), C(join) = |R| + |R|.|S| + (sj . |R| . |S|). El último término de la fórmula es el costo de escritura en almacenamiento temporario.
- Empleo de índices: Si hay un índice para el atributo de condición de reunión en S, podemos tomar todos los registros de R y utilizar el índice en S: C(join) = |R| + (|R| . (x + sb)) + (sj . |R| . |S|) dónde x es la cantidad de niveles del índice y sb la cardinalidad de selección sobre la condición de reunión.. En el caso de acceder por un índice primario, sb se convierte en 1. Si hay un hash para una de las dos tablas sobre el atributo de reunión, se reemplaza (x + sb) por h, dónde h es el número medio de accesos necesarios para obtener un registro.
- Merge-Sort: si los archivos están ordenados sobre el atributo de reunión, el costo es C(join) = |R| + |S| + (sj . |R| . |S|). Ahora si hay que ordenar los archivos, se le agrega el costo aproximado, es decir, el costo queda como C(join) = |R| + |S| + (sj . |R| . |S|) + (|R| . $\log_2$|R|) + (|S| . $\log_2$|S|) .

Optimizando las operaciones de una base de datos

Para optimizar la performance de un sistema, cualquiera sea su naturaleza, conviene comenzar estableciendo metas claras. No es lo mismo optimizar una base de datos OLTP que una aplicación de datawarehouse, ya que un datawarehouse tiene una correlación muy importante con las operaciones en paralelo y con manejo de grandes volúmenes de datos. A su vez, un sistema OLTP tiene una correlación importante con operaciones seriales (no paralelas) y un volumen importante de transacciones. Por lo tanto podríamos decir que estas aplicaciones tienen metas contrastantes, ya que para optimizar comandos SQL seriales, podríamos decir que la meta es minimizar la utilización de recursos (por la operación), y que para operaciones en paralelo, conviene maximizar el throughput.

Optimizando un comando SQL serial

Para esto, se puede explorar la performance de SQLs alternativos (sin cambiar la naturaleza del sistema) mediante el comando Explain Plan o similares según el DBMS, evaluando su plan de ejecución y su costo.

Optimizando operaciones en paralelo

Si se posee un sistema poderoso que procesa grandes volúmenes de datos, seguramente se querrá paralelizar los comandos, para que utilicen todos los recursos disponibles. Generalmente los DBMS pueden realizar las siguientes operaciones en paralelo:

- Consulta en paralelo
- DML paralelo
- DDL paralelo
- Recuperación en paralelo
- Carga masiva de datos en paralelo

En particular cuándo se quiere optimizar este tipo de plataformas, hay que prestar atención a las siguientes situaciones:

- Tiempo de ejecución largo: intentar paralelizar la operación lo más posible.
- Gran cantidad de datos: intentar dividir los registros de modo de poder procesar los sets de registros en distintos procesadores.

Optimizando aplicaciones OLTP

Generalmente para optimizar este tipo de aplicaciones, hay que optimizar comandos SQL seriales.

Para minimizar el proceso de parsing, se pueden utilizar variables ligadas (bound) en los comandos SQL desde las aplicaciones OLTP. De ese modo muchos usuarios pueden compartir la misma sentencia con distintos parámetros, ya que la misma permanecerá en el DMBS compilada y lista para ser utilizada.

Optimizando aplicaciones Datawarehouse

Para la optimización de este tipo de aplicaciones, por el contrario que en el modelo OLTP, se recomendará no utilizar variables ligadas (bound) en queries, ya que el optimizador podrá mediante el uso de literales determinar en mejor forma el plan de ejecución.

Metodología para optimizar las operaciones de una base de datos

1. Determinar cuáles son los queries que consumen la mayoría de los recursos
    a. Los que consumen grandes cantidades de recursos
    b. Los que consumen grandes cantidades de recursos por registro
    c. Los más ejecutados
2. Optimizar esos queries para que utilicen menos recursos
    a. Tratando de que utilicen menos recursos cuándo son ejecutados
    b. O usando dichos comandos menos frecuentemente.

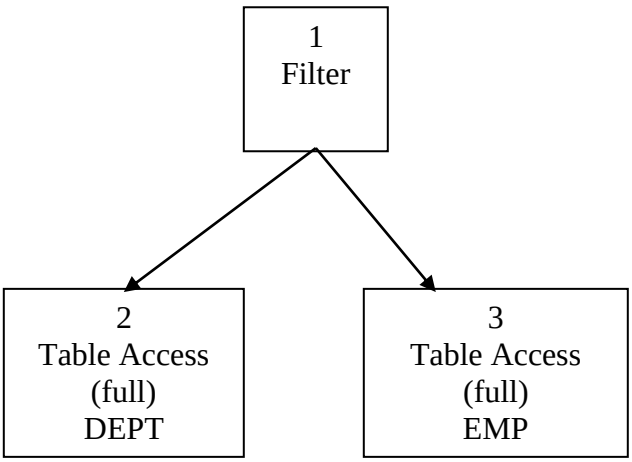
Enfoques para la optimización de comandos SQL

1. *Reestructurar los índices*: Esta actividad es siempre un buen punto de partida, ya que es de gran impacto en la performance. No hay que creer que los índices son la panacea, ya que un índice bien creado la mejorará, pero 50 índices sobre una tabla tienen muy poca probabilidad de mejorarla.
    a. Eliminar índices innecesarios
    b. Indexar los accesos críticos
    c. Considerar cluster teniendo en cuenta la unicidad

2. *Reestructurar los comandos SQL*: Después de reestructurar los índices, se puede pasar a la reestructuración de los comandos. Muchas veces es más fácil reescribir SQL que repararlo. Probablemente si se sabe la esencia del comando, se podrá escribir otro equivalente. Se pueden utilizar los resultados del comando Explain plan para comparar los planes de ejecución y costos de dos o más comandos para determinar cuál es el más eficiente.
   Ejemplo: Mostraremos dos comandos SQL que retornan todos los departamentos en la tabla DEPT que no tienen empleados en la tabla EMP. Cada comando busca en la tabla EMP en un subquery. Asumamos que hay un índice, DEPTNO_INDEX, en la columna DEPTNO de la tabla EMP:

```
select   DNAME, DEPTNO
from     DEPT
where    DPTNO not in
         (select   DEPTNO
         from      EMP);
```

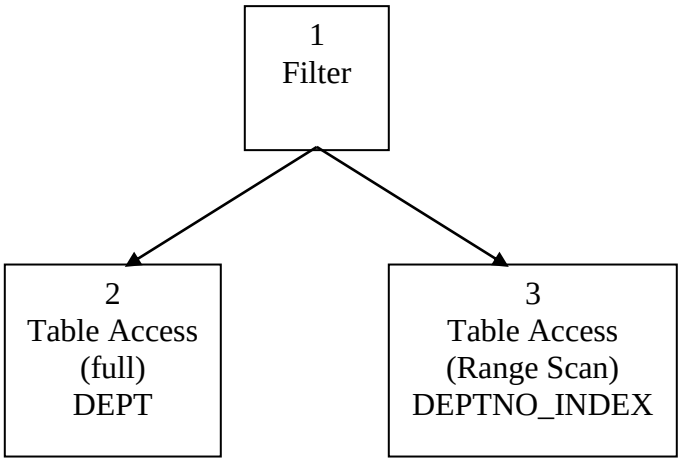
El plan de ejecución para el query es el siguiente:



En el paso 3 vemos que el DBMS realiza un full scan de la tabla EMP en lugar de utilizar el índice que se ha creado. Una operación de full scan consume muchos recursos. Evidentemente el DBMS no utiliza el índice creado a tal fin porque el subquery en cuestión no tiene una cláusula WHERE que lo indique. Ahora, si reescribimos el query de esta forma:

```
select   DNAME, DEPTNO
from     DEPT
where    not exists
         (select   DEPTNO
         from      EMP
         where     DEPT.DEPTNO = EMP.DEPTNO);
```

podemos ver que el plan de ejecución será similar a lo siguiente:



La cláusula WHERE en el query hizo que el paso 3 mute de un full scan a un range scan vía el uso de un índice.

Podríamos resumir ciertas reglas genéricas de optimización:

- Si en la aplicación tenemos queries que utilicen el IN, podríamos pensar en migrarlos a EXISTS si es posible.
- Usar valores de columnas sin transformar, por ejemplo:
  - where Month(Fecha)=8 and Year(Fecha)=2007 es peor que
  - where Fecha between '2007-08-01' and '2007-08-31'
- Evitar los casteos explícitos (evitan que se entre por índice)
- Escribir SQLs separados para valores específicos: Vale decir, quizás logramos un único query que hace todo lo que necesitamos, pero deberíamos evaluar con Explain Plan si dos queries con UNION no son mejores que nuestro query gigantesco.
- Tener cuidado cuándo hay una lista de valores en la cláusula IN, ya que muchas veces, dicha lista puede indicarnos que el diseño de la base de datos es pobre y que faltan entidades.
- Tener mucho cuidado cuándo se hace inner y outer join a views, sobre todo en views complejas, siempre evaluar el plan de acceso.
- No reciclar vistas: Si una vista fue creada con un objetivo específico, generalmente no es una buena idea utilizarlas para otros objetivos. Siempre cotejar el plan de ejecución.
- Elegir un orden de joins ventajosos: En muchos DBMS, el orden en el que escribimos los joins puede afectar la performance, esto lleva a un par de reglas genéricas:

- Evitar full table scans
- Evitar usar un índice que traiga 10.000 de la driving table si se puede usar otro que traiga 100.
- Seleccionar el orden de join que más convenga, es decir, que junte menos registros en los últimos estados del join order.

Tomemos el siguiente ejemplo:

```
select   STUFF
from     A, B, C
where    A.AC  between :ADESDE and :AHASTA
and      B.BC  between :BDESDE and :BHASTA
and      C.CC  between :CDESDE and :CHASTA
and      A.K1 = B.K1
and      A.K2 = C.K2;
```

1. Elegir la driving table y el driving index (si es posible). En general la driving table debe ser la que contenga la condición de filtro que elimine la mayor cantidad de datos posibles. Esto se hace sobre las condiciones de selección y no de junta.
2. Elegir los índices correctos. Una vez seleccionada la driving table, seleccionar el índice más selectivo disponible para ser utilizado. Alternativamente se puede elegir un full table scan si eso será más eficiente (pocos registros en dicha tabla).
3. Seleccionar el mejor orden de joins, dejando los mejores filtros antes. Dependiendo de las condiciones de filtro convendrá cambiar el orden de join o dejarlo cómo está.

3. *Reestructurar los datos*: Luego de realizar los dos pasos anteriores, puede considerarse la reestructuración de los datos:
    a. Introducir valores derivados al esquema, evitando los comandos GROUP BY en código que necesita una pronta respuesta. (denormalizar)
    b. Implementar entidades faltantes y sus tablas de intersección
    c. Reducir la carga de la red:
        i. Migrando
        ii. Replicando
        iii. Particionando
        iv. Distribuyendo

Optimización por costo

En general, siempre debería optimizarse por costo, ya que es un método soportado por casi todos los motores de base de datos, en cambio la optimización por regla no sólo no es aceptada por todos los DBMS sino que muchas veces se la pone en duda cómo metodología sustentable. También este método elimina la necesidad de optimizar sentencias SQL con hints. Para poder estimar de forma eficiente los parámetros necesarios para calcular costos, en el catálogo de la base de datos se mantienen histogramas, para esto hay que colectar dichas estadísticas de los objetos que queremos que funcionen de manera óptima.

Algo curioso para tomar en cuenta es que el optimizador en modo costo va a arrojar resultados totalmente distintos en un ambiente de desarrollo (pocos datos) que en uno productivo (masividad).

Los histograma tienen una cantidad estándar de buckets (inicial) que puede modificarse por configuración. Por ejemplo Oracle inicia sus histogramas con 75 buckets, ya que se cree que este valor es apropiado para la mayoría de las distribuciones de datos. Generalmente se tiene cuidado con cada histograma crítico, ya que la distribución no uniforme de los datos suele ser un problema.

Un ítem que debe ajustar el administrador de la base de datos es cada cuánto y de qué se corren estadísticas, ya que el resultado óptimo siempre es con los cálculos on-line, pero esto evidentemente sería carísimo de mantener y es por eso que una estimación sirve. Si los datos de determinada tabla cambian muy frecuentemente, habrá que planificar cuándo se tomará estadísticas de este objeto para que la optimización por costo no sea contraproducente (costo-beneficio).

Muchas bases de datos además de proveer el mecanismo clásico de computar con exactitud las estadísticas, pueden generar estimaciones basadas en técnicas de azar, en general se recomienda utilizar estas últimas técnicas, ya que el cálculo exacto consume muchos recursos (más teniendo en cuenta que se debe ejecutar periódicamente). Para tener una idea del consumo del cálculo, se puede tener en cuenta que para computar estadísticas de una tabla, el cálculo exacto demanda el tiempo de un full scan más el de ordenar todos los registros (por el atributo que se busca computar).

Cabe destacar que el DBA no debe de perder de vista el espacio de almacenamiento necesario para el catálogo (puede crecer drásticamente).

Usando optimización basada en reglas

Debería utilizarse solamente cuándo no sea posible recolectar estadísticas, o cuándo se comienza a correr una aplicación nueva, puede ser útil si no se confía en las estadísticas. Oracle incorpora este método pero recomienda migrar al método de costos.

Bibliografía y referencias

- "Sistemas de Bases de Datos - Conceptos fundamentales" - Elmasti/Navathe - Segunda Edición Adison Wesley Iberoamericana.
- "Oracle Performance Tuning Handbook" – Oracle Inc.